AI ASSISTANT CODING

ASSIGNMENT -11.3

TASK: Lab 11: Data Structures with AI Implementing Fundamental Data Structures using AI Assistance

Lab Objectives:

By the end of this lab, students will be able to:

Week6 -

Wednesday

• Design and implement fundamental data structures in Python using AI assistance.

• Effectively prompt AI tools (e.g., GitHub Copilot) for code generation, optimization, and documentation.

• Understand and compare core data structures: Arrays, Linked Lists, Stacks, Queues, Priority Queues, Trees, and Graphs.

• Improve code readability, efficiency, and maintainability using AI-generated suggestions.

Learning Outcomes

After completing this lab, students will be able to:

• Apply appropriate data structures to solve real-world problems.

• Analyze time and space complexity of different data structure operations.

• Use AI tools responsibly to assist (not replace) logical thinking and problem-solving.

• Validate, test, and refine AI-generated code.

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support

efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).

2. Implement the same functionality using a linked list for dynamic

memory allocation.

3. Implement the following operations in both approaches:

o Add a contact

o Search for a contact

o Delete a contact

4. Use GitHub Copilot to assist in generating search and delete methods.

5. Compare array vs. linked list approaches with respect to:

o Insertion efficiency

o Deletion efficiency

Expected Outcome

• Two working implementations (array-based and linked-list-based).

• A brief comparison explaining performance differences.

Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit

requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.

2. Extend the system to a Priority Queue, prioritizing faculty requests.

3. Use GitHub Copilot to assist in generating:

o enqueue() method

o dequeue() method

4. Test the system with a mix of student and faculty requests.

Expected Outcome

• Working queue and priority queue implementations.

• Correct prioritization of faculty requests.

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.

2. Provide the following operations:

o push(ticket)

o pop()

o peek()

3. Simulate at least five tickets being raised and resolved.

4. Use GitHub Copilot to suggest additional stack operations such as:

o Checking whether the stack is empty

o Checking whether the stack is full (if applicable)

Expected Outcome

• Functional stack-based ticket management system.

• Clear demonstration of LIFO behavior.

Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

• Insert

• Search

• Delete

Starter Code

class HashTable:

pass

Expected Outcome

• Collision handling using chaining

• Well-commented methods

Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

• Student Attendance Tracking

• Event Registration System

• Library Book Borrowing

• Bus Scheduling System

• Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.

2. Justify your choice in 2–3 sentences.

3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

• Mapping table: Feature → Data Structure → Justification

• One fully working Python implementation

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

CODE: from collections import deque

import heapq

```
# Lab 11: Data Structures with AI Assistance
# All Tasks Implementation

#
================================================================
======
# TASK 1: Smart Contact Manager (Arrays & Linked Lists)
#
================================================================
======

class ContactManagerArray:
    """Contact Manager using Array (List) approach"""

    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        """Add a contact to the array"""
        self.contacts.append({"name": name, "phone": phone})
        print(f"Contact '{name}' added successfully.")

    def search_contact(self, name):
        """Search for a contact by name"""
        for contact in self.contacts:
            if contact["name"].lower() == name.lower():
                return contact
        return None
```

```
    def delete_contact(self, name):
        """Delete a contact by name"""
        for i, contact in enumerate(self.contacts):
            if contact["name"].lower() == name.lower():
                self.contacts.pop(i)
                print(f"Contact '{name}' deleted successfully.")
                return True
        print(f"Contact '{name}' not found.")
        return False

    def display_all(self):
        """Display all contacts"""
        if not self.contacts:
            print("No contacts available.")
            return
        for contact in self.contacts:
            print(f"Name: {contact['name']}, Phone: {contact['phone']}")

class Node:
    """Node class for Linked List"""

    def __init__(self, name, phone):
        self.contact = {"name": name, "phone": phone}
        self.next = None

class ContactManagerLinkedList:
    """Contact Manager using Linked List approach"""
```

```
def __init__(self):
    self.head = None

def add_contact(self, name, phone):
    """Add a contact to the linked list"""
    new_node = Node(name, phone)
    if not self.head:
        self.head = new_node
    else:
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node
    print(f"Contact '{name}' added successfully.")

def search_contact(self, name):
    """Search for a contact by name"""
    current = self.head
    while current:
        if current.contact["name"].lower() == name.lower():
            return current.contact
        current = current.next
    return None

def delete_contact(self, name):
    """Delete a contact by name"""
    if not self.head:
```

```
            print("Contact list is empty.")
            return False

        if self.head.contact["name"].lower() == name.lower():
            self.head = self.head.next
            print(f"Contact '{name}' deleted successfully.")
            return True

        current = self.head
        while current.next:
            if current.next.contact["name"].lower() == name.lower():
                current.next = current.next.next
                print(f"Contact '{name}' deleted successfully.")
                return True
            current = current.next

        print(f"Contact '{name}' not found.")
        return False

    def display_all(self):
        """Display all contacts"""
        if not self.head:
            print("No contacts available.")
            return
        current = self.head
        while current:
            print(f"Name: {current.contact['name']}, Phone: {current.contact['phone']}")
            current = current.next
```

```
#
================================================================================
======
# TASK 2: Library Book Search System (Queues & Priority Queues)
#
================================================================================
======


class Queue:
    """Simple Queue (FIFO) for book requests"""

    def __init__(self):
        self.requests = deque()

    def enqueue(self, request):
        """Add a request to the queue"""
        self.requests.append(request)
        print(f"Request '{request}' added to queue.")

    def dequeue(self):
        """Remove and return the first request"""
        if not self.requests:
            print("Queue is empty.")
            return None
        return self.requests.popleft()

    def display(self):
        """Display all requests"""
```

```
        if not self.requests:
            print("Queue is empty.")
            return
        print("Queue:", list(self.requests))

class PriorityQueue:
    """Priority Queue for book requests (Faculty prioritized)"""

    def __init__(self):
        self.heap = []
        self.counter = 0

    def enqueue(self, request, priority):
        """Add a request with priority (0=Faculty, 1=Student)"""
        heapq.heappush(self.heap, (priority, self.counter, request))
        self.counter += 1
        print(f"Request '{request}' added with priority {priority}.")

    def dequeue(self):
        """Remove and return the highest priority request"""
        if not self.heap:
            print("Queue is empty.")
            return None
        priority, _, request = heapq.heappop(self.heap)
        return request

    def display(self):
        """Display all requests"""
```

```
        if not self.heap:
            print("Queue is empty.")
            return
        print("Priority Queue:", [(p, r) for p, _, r in self.heap])

#
================================================================
======
# TASK 3: Emergency Help Desk (Stack Implementation)
#
================================================================
======

class Stack:
    """Stack for managing support tickets"""

    def __init__(self, max_size=100):
        self.tickets = []
        self.max_size = max_size

    def push(self, ticket):
        """Add a ticket to the stack"""
        if len(self.tickets) >= self.max_size:
            print("Stack is full.")
            return False
        self.tickets.append(ticket)
        print(f"Ticket '{ticket}' pushed to stack.")
        return True
```

```
def pop(self):
    """Remove and return the top ticket"""
    if self.is_empty():
        print("Stack is empty.")
        return None
    return self.tickets.pop()

def peek(self):
    """View the top ticket without removing it"""
    if self.is_empty():
        print("Stack is empty.")
        return None
    return self.tickets[-1]

def is_empty(self):
    """Check if stack is empty"""
    return len(self.tickets) == 0

def is_full(self):
    """Check if stack is full"""
    return len(self.tickets) >= self.max_size

def display(self):
    """Display all tickets"""
    if self.is_empty():
        print("Stack is empty.")
        return
    print("Stack (Top to Bottom):", self.tickets[::-1])
```

```
#
==========================================================================
======
# TASK 4: Hash Table
#
==========================================================================
======


class HashTable:
    """Hash Table with chaining for collision handling"""

    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Hash function to calculate index"""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair"""
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                print(f"Updated: {key} = {value}")
                return
        self.table[index].append((key, value))
```

```
    print(f"Inserted: {key} = {value}")

def search(self, key):
    """Search for a value by key"""
    index = self._hash(key)
    for k, v in self.table[index]:
        if k == key:
            return v
    return None

def delete(self, key):
    """Delete a key-value pair"""
    index = self._hash(key)
    for i, (k, v) in enumerate(self.table[index]):
        if k == key:
            self.table[index].pop(i)
            print(f"Deleted: {key}")
            return True
    print(f"Key '{key}' not found.")
    return False

def display(self):
    """Display all entries"""
    for i, chain in enumerate(self.table):
        if chain:
            print(f"Index {i}: {chain}")
```

```
#
================================================================
======
# TASK 5: Real-Time Application Challenge
#
================================================================
======

class StudentAttendanceTracker:
    """Track student attendance using Hash Table"""

    def __init__(self):
        self.attendance = {}

    def mark_attendance(self, student_id, date, status):
        """Mark attendance for a student"""
        key = f"{student_id}_{date}"
        self.attendance[key] = status
        print(f"Attendance marked for {student_id} on {date}: {status}")

    def get_attendance(self, student_id, date):
        """Retrieve attendance status"""
        key = f"{student_id}_{date}"
        return self.attendance.get(key, "Not recorded")

    def display_all(self):
        """Display all attendance records"""
        for key, status in self.attendance.items():
            print(f"{key}: {status}")
```

```
class EventRegistrationSystem:
    """Event registration using Set data structure"""

    def __init__(self):
        self.registered = set()

    def register(self, student_id):
        """Register a student for event"""
        if student_id in self.registered:
            print(f"Student {student_id} already registered.")
        else:
            self.registered.add(student_id)
            print(f"Student {student_id} registered successfully.")

    def unregister(self, student_id):
        """Unregister a student"""
        if student_id in self.registered:
            self.registered.remove(student_id)
            print(f"Student {student_id} unregistered.")
        else:
            print(f"Student {student_id} not found.")

    def display_registered(self):
        """Display all registered students"""
        print(f"Registered students: {self.registered}")

class BookBorrowingSystem:
```

```
    """Library book borrowing using Stack"""

    def __init__(self):
        self.borrow_history = []

    def borrow_book(self, student_id, book_name):
        """Borrow a book"""
        self.borrow_history.append((student_id, book_name))
        print(f"{student_id} borrowed '{book_name}'")

    def display_recent_borrows(self, count=5):
        """Display recent borrow records"""
        print("Recent borrows:")
        for record in self.borrow_history[-count:]:
            print(f"  {record[0]}: {record[1]}")

class BusSchedulingSystem:
    """Bus scheduling using Queue"""

    def __init__(self):
        self.schedule = deque()

    def add_route(self, bus_id, departure, destination):
        """Add a bus route"""
        self.schedule.append((bus_id, departure, destination))
        print(f"Route added: Bus {bus_id} from {departure} to {destination}")

    def next_bus(self):
```

```
        """Get next bus in schedule"""
        if self.schedule:
            return self.schedule.popleft()
        return None

    def display_schedule(self):
        """Display all scheduled routes"""
        if not self.schedule:
            print("No routes scheduled.")
            return
        for bus_id, departure, destination in self.schedule:
            print(f"Bus {bus_id}: {departure} → {destination}")

class CafeteriaOrderQueue:
    """Cafeteria orders using Queue"""

    def __init__(self):
        self.orders = deque()

    def place_order(self, order_id, items):
        """Place an order"""
        self.orders.append((order_id, items))
        print(f"Order {order_id} placed: {items}")

    def process_order(self):
        """Process next order"""
        if self.orders:
            return self.orders.popleft()
```

```
        return None

    def display_orders(self):
        """Display pending orders"""
        if not self.orders:
            print("No pending orders.")
            return
        for order_id, items in self.orders:
            print(f"Order {order_id}: {items}")

#
==========================================================================
======
# MAIN EXECUTION AND TESTING
#
==========================================================================
======

if __name__ == "__main__":
    print("="*70)
    print("LAB 11: DATA STRUCTURES WITH AI ASSISTANCE")
    print("="*70)

    # TASK 1: Contact Manager
    print("\n--- TASK 1: CONTACT MANAGER (ARRAY vs LINKED LIST) ---\n")

    print("Array-based Contact Manager:")
    cm_array = ContactManagerArray()
    cm_array.add_contact("Alice", "555-0001")
```

```
cm_array.add_contact("Bob", "555-0002")
cm_array.add_contact("Charlie", "555-0003")
print("Search Bob:", cm_array.search_contact("Bob"))
cm_array.delete_contact("Bob")
cm_array.display_all()

print("\nLinked List-based Contact Manager:")
cm_ll = ContactManagerLinkedList()
cm_ll.add_contact("Alice", "555-0001")
cm_ll.add_contact("Bob", "555-0002")
cm_ll.add_contact("Charlie", "555-0003")
print("Search Bob:", cm_ll.search_contact("Bob"))
cm_ll.delete_contact("Bob")
cm_ll.display_all()

# TASK 2: Queue and Priority Queue
print("\n--- TASK 2: LIBRARY BOOK SYSTEM (QUEUE & PRIORITY QUEUE) ---\n")

print("Simple Queue (FIFO):")
queue = Queue()
queue.enqueue("Request 1")
queue.enqueue("Request 2")
queue.enqueue("Request 3")
queue.display()
print("Processing:", queue.dequeue())

print("\nPriority Queue (Faculty prioritized):")
pq = PriorityQueue()
```

```
pq.enqueue("Student Request 1", 1)
pq.enqueue("Faculty Request 1", 0)
pq.enqueue("Student Request 2", 1)
pq.enqueue("Faculty Request 2", 0)
pq.display()
print("Processing (highest priority):", pq.dequeue())

# TASK 3: Stack
print("\n--- TASK 3: EMERGENCY HELP DESK (STACK) ---\n")

stack = Stack()
tickets = ["Ticket#1: Login Issue", "Ticket#2: Printer Error",
        "Ticket#3: Network Down", "Ticket#4: Software Crash",
        "Ticket#5: Email Sync"]

print("Adding tickets:")
for ticket in tickets:
   stack.push(ticket)

stack.display()
print(f"\nTop ticket: {stack.peek()}")
print(f"Resolving: {stack.pop()}")
print(f"Is stack full? {stack.is_full()}")

# TASK 4: Hash Table
print("\n--- TASK 4: HASH TABLE ---\n")

ht = HashTable(5)
```

```
ht.insert("student_001", "Alice")
ht.insert("student_002", "Bob")
ht.insert("student_003", "Charlie")
print("Search student_001:", ht.search("student_001"))
ht.delete("student_002")
ht.display()

# TASK 5: Real-Time Application Challenge
print("\n--- TASK 5: CAMPUS RESOURCE MANAGEMENT ---\n")

print("Student Attendance Tracker (Hash Table):")
attendance = StudentAttendanceTracker()
attendance.mark_attendance("S001", "2024-01-15", "Present")
attendance.mark_attendance("S002", "2024-01-15", "Absent")

print("\nEvent Registration (Set):")
events = EventRegistrationSystem()
events.register("S001")
events.register("S002")
events.display_registered()

print("\nBook Borrowing (Stack):")
books = BookBorrowingSystem()
books.borrow_book("S001", "Python Basics")
books.borrow_book("S002", "Data Structures")
books.display_recent_borrows()

print("\nBus Scheduling (Queue):")
```

```
buses = BusSchedulingSystem()
buses.add_route("B1", "8:00 AM", "Main Campus")
buses.add_route("B2", "8:30 AM", "Annex")
buses.display_schedule()

print("\nCafeteria Orders (Queue):")
cafeteria = CafeteriaOrderQueue()
cafeteria.place_order("O1", "Sandwich + Coffee")
cafeteria.place_order("O2", "Salad + Juice")
cafeteria.display_orders()

print("\n" + "="*70)
print("END OF LAB 11")
print("="*70)
```

OUTPUT: